

tailguard

Build & Deploy Story

Two read-only tools that check a Tailscale network for the **"Tailworm"** risk,
and explain in plain English what's wrong and how to fix it.

🐧 Linux bash script · 🖥️ Windows .exe (x64 + ARM) · 🔒 100% read-only · 🧪 140 checks × 4 test runs · 📄 Plain-English reports

version 1.1.0 · October 2026 · built by **Anshuva** · open source (MIT)

Contents

Click any card to jump to that page. Every page has a ↩ **Contents** link at the bottom.

The problem — Tailworm in plain words — 3

What we built — Two tools, one result — 4

How it works — From tailscale CLI to report — 5

What it checks — 13 device checks + policy rules — 6

How it was built — Timeline, bugs caught on the way — 7

Testing — Fake Tailscale, 5 test suites — 8

Deploy & use — Windows, Linux, your policy — 9

Example results — A real home network, fixed in order — 10

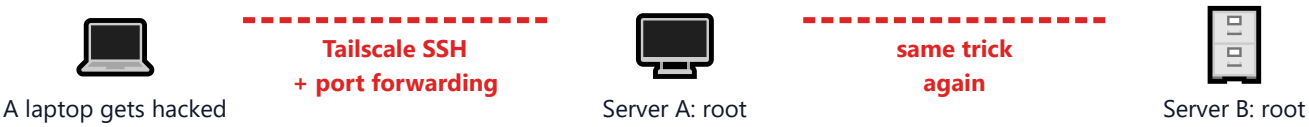
Changing it later — Recipes, rules, rebuild — 11

Back to cover — Start again — 1

> 💡 **In one sentence**
>
> tailguard looks at a Tailscale device and your access policy without changing anything, then tells you, in order of urgency, what to fix to stop one hacked device from taking over the rest.

🐛 The problem: "Tailworm"

On 25 October 2026 a security researcher showed how one hacked device can spread through a Tailscale network.



📰 What was reported

- "Tailworm" by Cyberdione Labs, reported by The Hacker News.
- With **Tailscale SSH** on and SSH **port forwarding** allowed, an attacker who controls one device can get **root** on any server the SSH rules let it reach.
- From there it can move on, device after device, like a worm.
- Tested on Tailscale 1.104.2 and headscale 0.29.5.

🩹 Patch status (Oct 2026)

- **No patch, no CVE, no security bulletin** yet.
- Older SSH bugs (such as TS-2026-009) were fixed in **1.98.9**, so anything older is worse.
- Tailscale hasn't confirmed the findings.

⚙️ So the fix is in your settings

Until a patch ships, protection comes from configuration: SSH rules that are narrow, no "allow everything" rules, Tailnet Lock on, Tailscale SSH off where it isn't needed, auto-updates on, and no stored login keys. That is exactly what tailguard checks.

🎯 The brief we built to

🔍 Detect

Find the risky settings on a device and in the exported access policy.

🔒 Don't touch

Strictly read-only. Never change a single setting.

🗣️ Explain

Every finding gets a status and a fix anyone can follow.

🛠️ What we built

Two tools with the same checks, the same wording and the same report format.

tailscale-audit.sh

- One bash file for any Linux: Debian/Ubuntu, RHEL/Fedora, Arch, Alpine.
- Reads JSON with `jq`, else `python3`, else a built-in awk parser, so it works even on a bare server.
- Searches `/etc`, systemd, cron, `/root` and home folders for saved keys.

tailscale-audit.exe

- Written in Go, standard library only: one portable file, no install, no DLLs.
- Builds for Windows x64 and ARM64, plus Linux for testing.
- Finds `tailscale.exe` on PATH or in Program Files. Live progress bar with a file counter.

📦 What's in the folder

FILE	WHAT IT'S FOR
`tailscale-audit.exe` / `-arm64.exe`	Run on Windows (as Administrator for the full check)
`tailscale-audit.sh`	Run on Linux (`sudo` for the full check)
`tailscale-audit-linux-amd64`	The Go tool built for Linux
`go/`	Go source code (about 1,800 lines)
`tests/`	Fake Tailscale, sample devices and policies, test runners
`README.md` · `CHANGELOG.md` · `LICENSE`	How to use it · what changed · MIT licence
`docs/` + this PDF	The logo and the source for this document

🎛️ Options

tailscale-audit --help

```
--policy FILE   also check your exported access policy (HuJSON)
--out DIR       where to save the .txt and .json reports
--no-color      plain text output
--no-progress   hide the progress bar
--help          show help

Exit code: 0 = no FAIL · 1 = at least one FAIL · 2 = the tool itself failed
```

🧭 How it works

It only asks questions. Tailscale answers, tailguard explains.

📡 Ask Tailscale
5 read-only commands:

```
version
status --json
debug prefs
serve status --json
lock status
```

→

🔎 Read the answers
Parse the real JSON fields like *RunSSH*, *ControlURL*, *KeyExpiry*, *sshHostKeys*. Search disk for saved *tskey-* keys.

→

⚖️ Judge
13 device checks + the policy rules. Each gets PASS, WARN, FAIL, INFO or UNKNOWN.

→

🗣️ Explain
Console + .txt + .json: problem, why it matters, how to fix, next steps.

🚫 Never guesses
Field names were taken from real Tailscale output on this PC. If a command or field is missing, the answer is **UNKNOWN**, never a fake PASS.

🔒 Never changes anything
No `set`, `up` or `down`, ever. The tests run a fake Tailscale that refuses any other command and logs every call.

🙈 Never shows a key
Any saved login key is cut to its first 10 characters: `tskey-auth****(masked)`.

⏱️ Never pretends
If the key search runs out of time in a big folder and finds nothing, it says **could not check** instead of "all clear".

📜 Policy check: honest about limits

The tool reads your exported policy (comments and trailing commas are fine) and describes each SSH rule as a sentence. Tailscale's documentation has **no documented policy setting to turn off SSH port forwarding**. Reports say the hosted service has a per-rule switch, but it isn't documented yet. So instead of inventing a setting, tailguard says so, recommends narrow rules, and checks the host-level off-switch `TS_SSH_DISABLE_FORWARDING` (in Tailscale's source, undocumented).

✅ What it checks

13 checks on the device, plus rules for the access policy.

#	CHECK	BAD RESULT	WHY IT MATTERS
C01	🔢 Tailscale version	FAIL below 1.98.9 · WARN otherwise	Old SSH bugs; Tailworm still unpatched
C02	🔌 Running and logged in	FAIL stopped / logged out	Other checks need it
C03	🖥️ Tailscale SSH on this device	WARN if on	The door Tailworm walks through
C04	🏢 Control server	WARN headscale	Forwarding can't be turned off from the policy
C05	🔐 Tailnet Lock	FAIL if off	Stops rogue devices being added
C06	⏳ Key expiry	WARN never expires	Stolen devices keep access forever
C07	🔄 Auto-updates	WARN if off	The fix must install itself
C08	🧦 Serve to Unix sockets	FAIL any `unix:`	e.g. docker.sock = full control
C09	🔀 App connector · routes · exit node	WARN connector	Reported way around access rules
C10	🔑 Saved login keys on disk	WARN found	Anyone reading them can join
C11	🎯 Devices accepting SSH	INFO count	How far an attack could spread
C12	🛡️ Admin rights	INFO	Lists anything skipped
C13	🔌 SSH forwarding off-switch	WARN SSH on without it	Blocks the tunnels Tailworm uses (Linux)

📜 Policy rules (`--policy`)

RULE	RESULT	FIX
SSH rule is broad: `*`, `autogroup:member`, any `group:`, or several sources/destinations	FAIL	One person → one device per rule
One person → one `tag:`	WARN	Make sure only one device has the tag
SSH rule allows `root`	WARN	Normal account + sudo
SSH rule uses `accept`	WARN	Use `check` mode
No documented policy setting blocks port forwarding	WARN	Narrow rules + forwarding off-switch (C13)
App connectors in `nodeAttrs` or `grants`	WARN	Remove if unused
Allow-everything ACL or grant	WARN	Specific rules + "tests" section

How it was built

Built on 26 Oct 2026, reviewed and improved on 27 Oct. Every change was planned, approved, then built and tested.

Investigate (read-only)

Checked the PC: Tailscale 1.102.3, Git Bash, no Go. Captured the real JSON from the 4 Tailscale commands. Researched Tailworm and searched the Tailscale docs and source for a port-forwarding setting.

Plan → approve

A written plan, approved before any file was created (the owner's standing rule).

Bash script

All checks, a HuJSON reader, and three JSON parsers (jq, python3, awk) with identical output.

Go tool

The same checks in Go, stdlib only; Windows admin detection; colour console.

Fake Tailscale + tests

Risky, safe and "unknown" devices; bad, mixed and good policies; expected results for each.

Toolchain on approval

The first download was blocked by a safety check. After the owner allowed it, portable Go, shellcheck and jq were downloaded (checksum verified) and deleted after each build.

First real run: 2 bugs found and fixed

The Windows key search timed out but still said PASS → now per-folder limits and UNKNOWN. "0 of 0 devices" shown while disconnected → now UNKNOWN. Shellcheck also caught a test-runner bug.

Progress bar

The owner saw a silent window, so: a live step bar, spinner and file counter, shown on screen only.

Plain-English report

Rebuilt the output so anyone understands it: problems first, why it matters, step-by-step fixes, next steps, and bright colours for the blue PowerShell window.

Docs

README, CHANGELOG and this PDF.

Published on GitHub: v1.0.0

Open source under MIT, with Windows and Linux downloads. A leak scan of every published file made sure no private details went public.

Outside review → v1.1.0

A reviewer found real gaps: no check for the SSH forwarding off-switch, fix advice that failed its own tag rule, "auto-update off" when it was only unset, and no time limit on the Linux key search. All fixed, now 140 tests per run.

🧪 Testing

A fake Tailscale lets every situation be tested on any PC, safely and repeatably.

TEST RUNS
4 × 140
bash (awk, jq, python) and the .exe, every check passing

SAME WORDING
7 / 7
bash and .exe print the same report in every case

STATIC CHECKS
0 issues
go vet (Win x64, ARM64, Linux), go test, shellcheck

🤖 The test devices

🔥 risky
Old version, SSH on, headscale, no lock, docker.sock shared, app connector, exit node, two planted fake keys.

🌱 safe
Current version, everything set correctly. Should be all PASS (plus the unpatched-Tailworm WARN).

❓ unknown
Odd version text, missing commands and fields. Everything must be UNKNOWN, never PASS.

✅ What the tests prove

1. Each check gets the exact expected status on each device, and each policy gets **exactly** the expected findings (nothing missing, nothing extra).
2. Only the 5 read-only Tailscale commands are ever run. The fake refuses anything else.
3. No login key ever appears unmasked in the console, the .txt or the .json.
4. Every problem has a plain title, an explanation and at least one fix step.
5. Exit codes are right: 0 clean, 1 with a FAIL, 2 for a bad flag or a broken policy file.
6. Both tools print the same words (after normalising `sudo` and file paths).

Git Bash

```
sh-awk     passed: 140  failed: 0
sh-jq      passed: 140  failed: 0
sh-python  passed: 140  failed: 0
exe        passed: 140  failed: 0
compare    passed: 7  failed: 0
```

🚀 Deploy & use

There's no server and nothing to install: copy one file to a device and run it.

🏠 This PC

Open **PowerShell as Administrator** (Start → PowerShell → Run as administrator), then run the command below. The report is saved next to the exe.

💻 Other Windows PCs

Copy `tailscale-audit.exe` (or `-arm64.exe` on ARM) to the PC and run it as Administrator. No install, no DLLs.

🐧 Linux servers

Copy `tailscale-audit.sh` over and run it with `sudo`. Works without jq or python.

Administrator: Windows PowerShell

```
PS> .\tailscale-audit.exe --policy .\policy.hujson
[##########------] 6/7 / Scanning for auth keys  0:42  | 18240 files
```

Linux

```
$ scp tailscale-audit.sh policy.hujson server:
$ ssh server sudo bash tailscale-audit.sh --policy policy.hujson
```

📜 Get your policy file (optional, recommended)

1. Open login.tailscale.com/admin/acls (Access controls).
2. Switch to the **JSON editor**, click inside, press **Ctrl+A** then **Ctrl+C**. Don't save anything there.
3. Paste into a new file `policy.hujson` and pass it with `--policy`.

👀 Reading the report

[X]	[!]	[?]	[OK]	[i]
Fix now	Review	Couldn't check	Fine	For info

Each problem card shows **Problem**, **Why it matters**, lettered **How to fix** steps with exact commands, and a **More help** link. The **NEXT STEPS** list at the end is your to-do list.

Example results

A real Administrator run on a home network with the default Tailscale policy, October 2026.

FIX NOW	REVIEW	OK	DEVICES WITH SSH
2	4	8	none

Good news first

No device on the network accepts Tailscale SSH and the PC has it off, so Tailworm had nothing to attack. Auto-updates were on, with no risky shares, no saved keys and normal key expiry. The problems were all in the policy and Tailnet Lock, which is typical for a default setup.

Fix plan, most important first

#	WHAT		HOW
1	**Turn on Tailnet Lock**	FIX NOW	`tailscale lock init --gen-disablements 1 --confirm tlpub:KEY1 tlpub:KEY2` with keys from 2+ trusted devices; keep the disablement secret offline.
2	**Delete or narrow SSH rule 1**	FIX NOW	Nothing uses Tailscale SSH, so deleting the rule removes the risk. It's Tailscale's default (own devices only), so lower risk, but it's still flagged.
3	**Remove root from SSH rule 1**	REVIEW	If you keep the rule: `"users": ["autogroup:nonroot"]`.
4	**Replace the allow-everything grant**	REVIEW	Start from "own devices only" and add grants per server. Use a "tests" section and Preview first.
5	**Keep SSH rules narrow**	REVIEW	No documented policy setting blocks port forwarding; on Linux servers turn on the forwarding off-switch (C13).
6	**Watch for the Tailworm fix**	REVIEW	Auto-update is on already; check security bulletins.

Then

Run the tool again as Administrator. When the RESULT line shows no problems to fix, you're done.

Changing it later

tailguard is open source (MIT). Here's how to change it, test it and contribute.

Easy first changes

- Add a check: give it the next ID (C14) in both tools
- Improve the wording: keep both tools word-for-word identical
- Check SSH agent forwarding too
- Add a test case under `tests/`

Rules every change keeps

- Read-only: only 5 Tailscale commands
- Missing data = UNKNOWN, never PASS
- No invented policy fields
- Keys always masked

Build and test

1. Install Go 1.22+, shellcheck and (optional) jq.
2. `bash build.sh` builds the binaries and the test fake.
3. Run the five test suites from the README: expect 140 passed four times and 7 for compare.
4. Open a pull request. Same checks, same words, both tools.

Quick start

```
$ bash build.sh
$ bash tests/run-tests.sh exe ./tailscale-audit.exe tests/fakets/tailscale.exe
passed: 140  failed: 0
```

> **tailguard**
> Read-only. Plain English. Same answer on Windows and Linux.